

Introducción a Python, R y julia

Santiago Gallego

2026-05-07

Contenido

0.1	Ejercicio 1: Hidrología del Río Magdalena (Vectores y Matemáticas)	2
0.1.1	python	2
0.1.2	R	2
0.1.3	python	2
0.1.4	R	3
0.1.5	python	3
0.1.6	R	3
0.1.7	python	4
0.1.8	R	4
0.1.9	python	4
0.1.10	R	4
0.2	Ejercicio 2: Calidad del Aire en Bogotá (DataFrames y Filtrado)	5
0.2.1	Python	5
0.2.2	R	6
0.2.3	Python	7
0.2.4	R	8
0.2.5	Python	8
0.2.6	R	9
0.3	Preguntas	10
0.3.1	Sobre el Ejercicio 1 (Vectorización):	10
0.3.2	Pregunta General (Indexación):	11

0.1 Ejercicio 1: Hidrología del Río Magdalena (Vectores y Matemáticas)

Contexto: Estás analizando el comportamiento de las estaciones hidrológicas del IDEAM a lo largo del Río Magdalena. Has recibido el reporte del caudal medio mensual (en metros cúbicos por segundo) de cinco estaciones clave y necesitas extraer estadísticos básicos y transformar las unidades para un informe ambiental.

Instrucciones de código:

1. Define una colección ordenada (Vector/Lista/Array) llamada estaciones con los siguientes nombres: “Honda”, “Puerto Berrío”, “Barrancabermeja”, “Puerto Wilches”, “Calamar”.

0.1.1 python

```
import numpy as np

# [1] DEFINICIÓN DE NOMBRES (Colección ordenada)
# Usamos una lista estándar porque son datos de texto (strings)
estaciones = ["Honda", "Puerto Berrío", "Barrancabermeja", "Puerto Wilches", "Calamar"]
```

0.1.2 R

```
# [1] DEFINICIÓN DE NOMBRES
# Usamos c() (combine) para crear un vector de caracteres
estaciones <- c("Honda", "Puerto Berrío", "Barrancabermeja", "Puerto Wilches", "Calamar")
```

2. Define otra colección paralela llamada `caudales_m3s` con los siguientes valores numéricos: 1500, 2100, 2800, 3200, 4500. Utilizando las funciones estadísticas nativas de tu lenguaje elegido, calcula e imprime: El caudal máximo registrado en el río. El caudal promedio de las cinco estaciones.

0.1.3 python

```
# [2] DEFINICIÓN DE CAUDALES (Colección paralela)
# Convertimos a np.array para habilitar funciones matemáticas de alto rendimiento
caudales_m3s = np.array([1500, 2100, 2800, 3200, 4500])
```

0.1.4 R

```
# [2] DEFINICIÓN DE CAUDALES
# Creamos un vector numérico. En R, los vectores son la unidad básica de datos
caudales_m3s <- c(1500, 2100, 2800, 3200, 4500)
```

3. Para un estudio local, necesitas el caudal en litros por segundo (l/s). Utilizando el concepto de vectorización (o broadcasting), multiplica la colección `caudales_m3s` por 1000 y guarda el resultado en una nueva variable llamada `caudales_ls`. (Asegúrate de no usar bucles for).

0.1.5 python

```
# [3] CÁLCULO DEL MÁXIMO
# Usamos .max() de numpy porque recorre el array a nivel de C (muy rápido)
caudal_max = caudales_m3s.max()
print(f"1. Caudal Máximo registrado: {caudal_max} m³/s")
```

1. Caudal Máximo registrado: 4500 m³/s

```
# [3] CÁLCULO DEL PROMEDIO
# .mean() suma todos los elementos y divide por el total en una sola operación
caudal_promedio = caudales_m3s.mean()
print(f"2. Caudal Promedio del río: {caudal_promedio} m³/s")
```

2. Caudal Promedio del río: 2820.0 m³/s

0.1.6 R

```
# [3] CÁLCULO DEL MÁXIMO
# La función max() es nativa y está optimizada para ignorar NAs si se le pide
caudal_max <- max(caudales_m3s)
cat("1. Caudal Máximo registrado:", caudal_max, "m3/s\n")
```

1. Caudal Máximo registrado: 4500 m3/s

```
# [3] CÁLCULO DEL PROMEDIO
# mean() es la función estándar. Es más eficiente que hacer (sum / length) manualmente
caudal_promedio <- mean(caudales_m3s)
cat("2. Caudal Promedio del río:", caudal_promedio, "m3/s\n")
```

2. Caudal Promedio del río: 2820 m3/s

4. Imprime la colección resultante caudales_ls.

0.1.7 python

```
# [4] TRANSFORMACIÓN VECTORIZADA (Sin ciclos for)
# El "Broadcasting" permite que el escalar 1000 se aplique a cada celda del array
# Esto evita la lentitud de un bucle 'for' de Python
caudales_ls = caudales_m3s * 1000
```

0.1.8 R

```
# [5] TRANSFORMACIÓN VECTORIZADA
# R aplica el "Reciclaje de vectores": multiplica el 1000 por cada elemento
# Es la forma más rápida de transformar unidades en R
caudales_ls <- caudales_m3s * 1000
```

0.1.9 python

```
print("\n3. Colección resultante (l/s):")
```

3. Colección resultante (l/s):

```
print(caudales_ls)
```

```
[1500000 2100000 2800000 3200000 4500000]
```

0.1.10 R

```
cat("\n3. Colección resultante (l/s):\n")
```

3. Colección resultante (l/s):

```
print(caudales_ls)
```

```
[1] 1500000 2100000 2800000 3200000 4500000
```

`np.array / c()`: Son los constructores. Sin ellos, los datos son solo valores sueltos; con ellos, son una estructura matemática.

`max() / mean()`: Son funciones de reducción. Toman muchos valores y los convierten en un solo estadístico.

`"*" 1000`: Es una operación de mapeo. Transforma cada valor individual del grupo original sin alterar la cantidad de elementos.

0.2 Ejercicio 2: Calidad del Aire en Bogotá (DataFrames y Filtrado)

Contexto: La Red de Monitoreo de Calidad del Aire de Bogotá ha publicado los datos promedio diarios de Material Particulado 2.5. Debes organizar estos datos en una estructura tabular, filtrar las estaciones que presentan riesgos para la salud (según la OMS) y visualizar los resultados.

1. Crea un DataFrame llamado `df_aire` (utilizando la librería adecuada según tu lenguaje: `pandas`, `data.frame` o `DataFrames.jl`) que contenga dos columnas: `estacion`: “Carvajal”, “Kennedy”, “Fontibón”, “Suba”, “Usaquén” `pm25`: 55, 42, 38, 15, 12
2. Imprime un resumen descriptivo/técnico de tu DataFrame (usando la función equivalente a `info()`, `str()` o `describe()`).

0.2.1 Python

```
import pandas as pd
import matplotlib.pyplot as plt # Importamos para la visualización posterior

# 1. Creación del DataFrame
datos = {
    "estacion": ["Carvajal", "Kennedy", "Fontibón", "Suba", "Usaquén"],
    "pm25": [55, 42, 38, 15, 12]
}
df_aire = pd.DataFrame(datos)

# 2. Resumen descriptivo
print("--- RESUMEN TÉCNICO (Python) ---")
```

--- RESUMEN TÉCNICO (Python) ---

```
print(df_aire.info()) # Muestra tipos de datos y valores no nulos
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  -
0   estacion    5 non-null      object
1   pm25        5 non-null      int64
dtypes: int64(1), object(1)
memory usage: 212.0+ bytes
None
```

```
print("\n--- ESTADÍSTICAS BÁSICAS ---")
```

--- ESTADÍSTICAS BÁSICAS ---

```
print(df_aire.describe()) # Muestra media, min, max, etc.
```

```
count    pm25
count    5.000000
mean     32.400000
std      18.392933
min      12.000000
25%      15.000000
50%      38.000000
75%      42.000000
max      55.000000
```

0.2.2 R

```
# 1. Creación del data.frame
df_aire <- data.frame(
  estacion = c("Carvajal", "Kennedy", "Fontibón", "Suba", "Usaquén"),
  pm25 = c(55, 42, 38, 15, 12),
  stringsAsFactors = FALSE # Evita que los nombres se traten como factores/categorías
)

# 2. Resumen descriptivo
cat("--- ESTRUCTURA DEL DATAFRAME (R) ---\n")
```

--- ESTRUCTURA DEL DATAFRAME (R) ---

```
str(df_aire) # Muestra la estructura interna del objeto
```

```
'data.frame':  5 obs. of  2 variables:
 $ estacion: chr  "Carvajal" "Kennedy" "Fontibón" "Suba" ...
 $ pm25    : num  55 42 38 15 12
```

```
cat("\n--- RESUMEN ESTADÍSTICO ---\n")
```

--- RESUMEN ESTADÍSTICO ---

```
summary(df_aire) # Proporciona un resumen de los valores numéricos
```

estacion	pm25
Length:5	Min. :12.0
Class :character	1st Qu.:15.0
Mode :character	Median :38.0
	Mean :32.4
	3rd Qu.:42.0
	Max. :55.0

3. El límite diario recomendado por la OMS para es de 15 . Crea un nuevo DataFrame llamado df_alerta que filtre y contenga únicamente las estaciones donde el nivel de pm25 sea estrictamente mayor a 15.
4. Crea una nueva columna en df_alerta llamada estado y asigne a todas sus filas el valor de texto "Crítico".

0.2.3 Python

```
# 3. Filtrado: Estaciones con PM2.5 > 15
# Usamos .copy() para crear un objeto independiente y evitar advertencias de memoria
df_alerta = df_aire[df_aire["pm25"] > 15].copy()

# 4. Nueva columna: Asignamos el valor "Crítico" a todas las filas filtradas
df_alerta["estado"] = "Crítico"

print("--- ESTACIONES EN ALERTA ---")
```

```
--- ESTACIONES EN ALERTA ---
```

```
print(df_alerta)
```

	estacion	pm25	estado
0	Carvajal	55	Crítico
1	Kennedy	42	Crítico
2	Fontibón	38	Crítico

0.2.4 R

```
# 3. Filtrado: Seleccionamos filas donde pm25 es estrictamente mayor a 15
df_alerta <- df_aire[df_aire$pm25 > 15, ]

# 4. Nueva columna: Creamos la columna 'estado' directamente
df_alerta$estado <- "Crítico"

cat("--- ESTACIONES EN ALERTA ---\n")
```

```
--- ESTACIONES EN ALERTA ---
```

```
print(df_alerta)
```

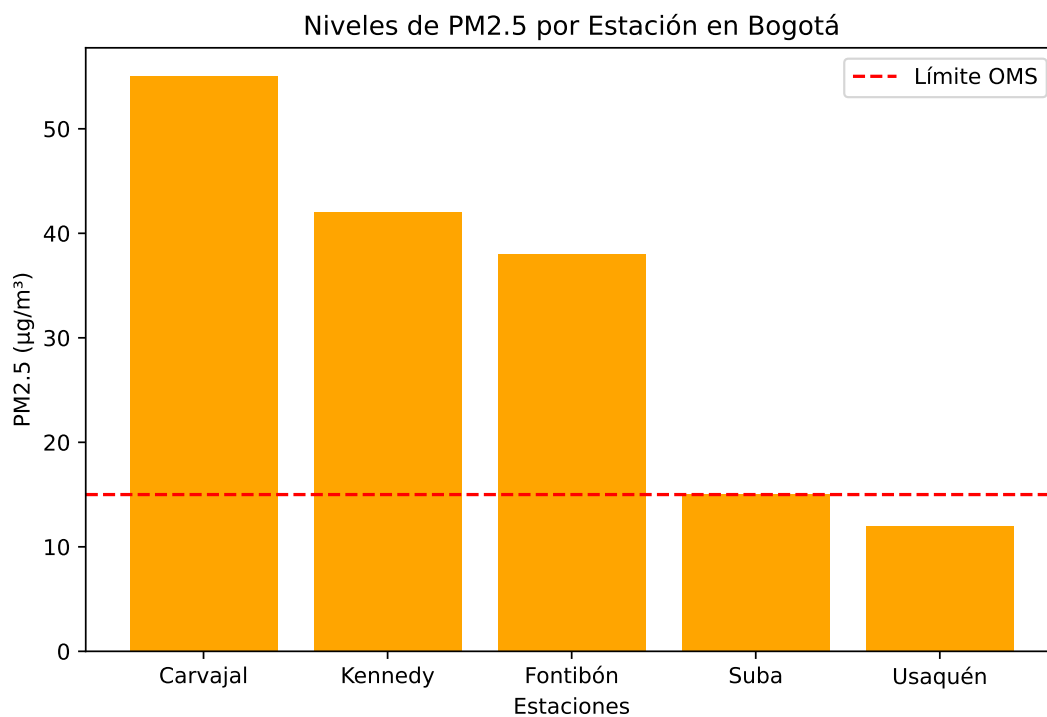
	estacion	pm25	estado
1	Carvajal	55	Crítico
2	Kennedy	42	Crítico
3	Fontibón	38	Crítico

- Utiliza la librería gráfica nativa de tu entorno para generar un gráfico de barras simple usando el DataFrame original (df_aire), donde el eje X sean las estaciones y el eje Y sean los niveles de PM25

0.2.5 Python


```
# 5. Gráfico de barras
plt.figure(figsize=(8, 5)) # Definimos el tamaño de la figura
plt.bar(df_aire["estacion"], df_aire["pm25"], color='orange') # Color naranja para advertencia

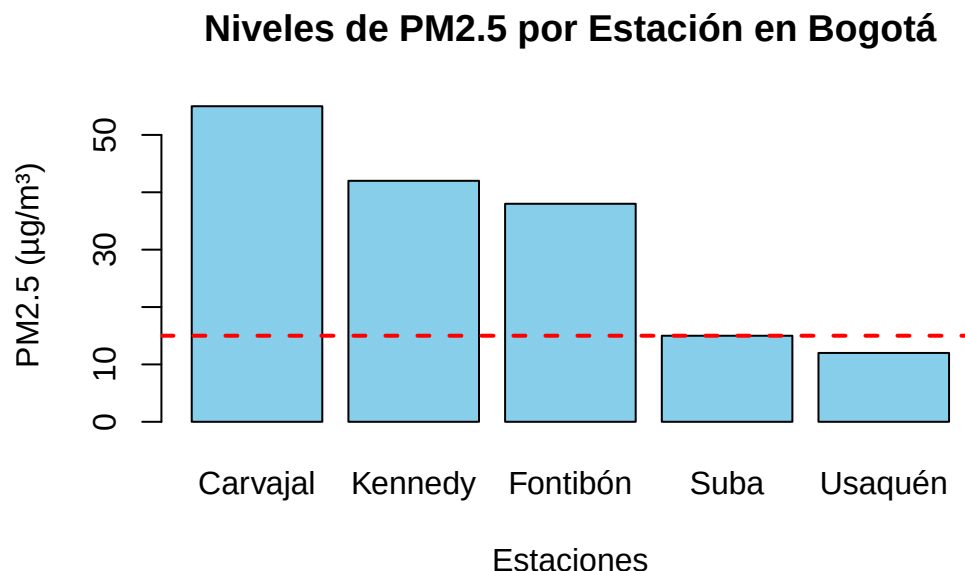
# Personalización
plt.axhline(y=15, color='red', linestyle='--', label='Límite OMS') # Línea de referencia
plt.title("Niveles de PM2.5 por Estación en Bogotá")
plt.xlabel("Estaciones")
plt.ylabel("PM2.5 (µg/m³)")
plt.legend()
plt.show()
```



0.2.6 R

```
# 5. Gráfico de barras nativo
barplot(df_aire$pm25,
        names.arg = df_aire$estacion,
        col = "skyblue",
        main = "Niveles de PM2.5 por Estación en Bogotá",
        xlab = "Estaciones",
        ylab = "PM2.5 (µg/m³)")
```

```
# Añadimos la línea de límite de la OMS (y = 15)
abline(h = 15, col = "red", lty = 2, lwd = 2)
```



0.3 Preguntas

0.3.1 Sobre el Ejercicio 1 (Vectorización):

Intenta explicar qué sucedería si decides hacer este ejercicio en Python y creas `caudales_m3s` como una Lista nativa (`[1500, 2100...]`) y la multiplicas directamente por 1000 (`caudales_m3s * 1000`). ¿Realizaría la operación matemática deseada? ¿Qué librería de Python soluciona este problema y cómo se diferencia este comportamiento del de R o Julia?

- No, no realizaría la operación matemática deseada. Si multiplicas una lista nativa de Python por un entero (por ejemplo, `[1, 2, 3] * 1000`), Python interpreta el operador `*` como una operación de replicación o repetición. El resultado sería una lista gigantesca que contiene los mismos 5 valores originales repetidos 1000 veces uno tras otro. En lugar de transformar el caudal de la estación Honda de 1500 a 1,500,000, tendrías una lista con 5000 elementos donde el número 1500 aparece mil veces. La solución: NumPy. La librería que soluciona este problema en Python es NumPy mediante su estructura principal: el `np.array`. NumPy introduce el concepto de Broadcasting (difusión). Al convertir la lista en un arreglo de NumPy, la librería redefine el comportamiento de los operadores aritméticos. Cuando NumPy ve `array * 1000`, entiende que debe “difundir” el escalar (1000) sobre cada elemento individual del vector, ejecutando la operación matemática elemento a elemento a nivel de lenguaje C (lo cual es extremadamente rápido).

Diferencia con R o Julia

A diferencia de Python, R y Julia fueron diseñados desde su origen para el cálculo numérico y estadístico, por lo que su comportamiento nativo es diferente: - **En R** La estructura básica es el vector. R no tiene “escalares” puros; el número 1000 es en realidad un vector de longitud 1. Por lo tanto, el lenguaje aplica de forma nativa el “Reciclaje de Vectores”, realizando la multiplicación elemento a elemento sin necesidad de librerías externas. - **En Julia** El lenguaje está diseñado para el alto rendimiento científico. Aunque permite operaciones vectorizadas directas, es famoso por su sintaxis de “Dot Operators” (operadores de punto). En Julia, usarías `caudales_m3s .* 1000`. Ese punto indica explícitamente que la operación debe aplicarse a cada elemento, combinando la facilidad de lectura con una velocidad idéntica al lenguaje C o Fortran.

En resumen: Mientras que en Python se depende de NumPy para “activar” el comportamiento matemático de los vectores, en R y Julia ese comportamiento es el estándar del lenguaje.

0.3.2 Pregunta General (Indexación):

Escribe la línea de código exacta que usarías en tu lenguaje elegido para extraer las tres primeras estaciones del DataFrame del Ejercicio 2 usando indexación por rangos (slicing). Explica brevemente si el límite superior del rango que escribiste se incluye o se excluye en el resultado final, justificando esto según el tipo de indexación (Base-0 vs Base-1) de tu lenguaje.

```
primeras_tres = df_aire.iloc[0:3]
```

- **Base-0:** Python utiliza una indexación basada en cero, lo que significa que el primer elemento está en la posición 0.
- **Límite superior:** En el slicing nativo de Python y en `.iloc`, el límite superior se excluye.
- **Justificación:** Al escribir `0:3`, Python toma los índices 0, 1 y 2. Esto es una convención de diseño que permite saber rápidamente cuántos elementos se están extrayendo restando los índices ($3 - 0 = 3$), facilitando el manejo de subconjuntos de datos en la memoria.

```
primeras_tres <- df_aire[1:3, ]
```

- **Base-1:** R es un lenguaje de indexación basada en uno, por lo que la primera fila se encuentra estrictamente en la posición 1.
- **Límite superior:** En R, el límite superior se incluye.
- **Justificación:** El operador `:` en R genera una secuencia completa. Al escribir `1:3`, R genera el conjunto $\{1, 2, 3\}$. Al ser un lenguaje diseñado por y para estadísticos, se prioriza que el rango escrito coincida exactamente con la selección física de los elementos, evitando la necesidad de “sumar uno” mentalmente para alcanzar el último registro deseado.